HOW-TO / 01

DatenCockpit

Von der Tabelle zum verlässlichen Bericht.

Bedienung · Aufbau · Datenbank · Veröffentlichung

Was macht das Projekt? Kleine Unternehmen erhalten Auftragslisten oft als Excel- oder CSV-Datei. Fehlende Angaben, doppelte Aufträge und ungültige Werte können daraus falsche Berichte machen. Das DatenCockpit macht diese Auffälligkeiten sichtbar, bevor sie in die Auswertung gelangen.

Der Ablauf: Datei einlesen → Daten prüfen → Auffälligkeiten korrigieren oder Zeilen ausschließen → ausdrücklich freigeben → Monatskennzahlen aus PostgreSQL anzeigen.

Was am Anfang passiert

Beim Öffnen ist noch kein neuer Import gespeichert. Die Website lädt zunächst nur Oberfläche und öffentliche Konfiguration. Wird im selben Tab ein noch gültiger Import-Schlüssel gefunden, versucht sie, den zugehörigen Bericht wieder zu laden. Eine neu ausgewählte Datei wird zuerst ausschließlich im Browser gelesen und geprüft.

Das Ergebnis

Ein Monatsbericht zeigt bezahlten Umsatz, offene Aufträge, offenes Auftragsvolumen und die Zahl freigegebener Aufträge. Hinzu kommen Diagramm, CSV-Export und eine Löschfunktion. Maßgeblich ist der **Auftragsmonat**; dies ist keine buchhalterische Umsatzrealisierung.

Projekt und Unterlagen

DatenCockpit öffnen

Quellcode auf GitHub

Stand: 22. Oktober 2026 · Manfred Linner · Portfolio-Demo mit fiktiven Daten.

Kontakt für Ihr Datenprojekt

Manfred Linner · Softwareentwicklung & Dienstleistungen
Schwabach bei Nürnberg · Remote deutschlandweit und international

info@mlinner-software.de · +49 160 7838773

Diese Anleitung beschreibt die aktive Lösung: GitHub Pages + Supabase. Netlify, Azure und die zuvor getestete Neon-Datenbank werden für das aktive Cockpit nicht benötigt.

HOW-TO / 02

Die Architektur auf einen Blick

1 GitHub → GitHub Pages

Quellcode im Repository · Branch main · Ordner / (root)

Liefert HTML, CSS, JavaScript und öffentliche Downloads.

HTTPS: Seite und Dateien

2 Browser / DatenCockpit

CSV/XLSX einlesen → Regeln prüfen → Zeilen bearbeiten/ausschließen

Erst nach ausdrücklicher Freigabe werden Aufträge übertragen.

Der Import-Zugriffsschlüssel bleibt im aktuellen Browser-Tab.

HTTPS + öffentlicher Publishable-Key; RPC mit Daten/Import-Schlüssel

3 Supabase / PostgreSQL

Data API → vier gezielt freigegebene SQL-Funktionen

Status · validierter Import · Monatsbericht · Löschen

Tabellen: datencockpit_batches 1 : n datencockpit_orders

RLS aktiv · direkte Tabellenzugriffe gesperrt · nur Token-Hash gespeichert

Wer übernimmt welche Aufgabe?

Baustein	Aufgabe
GitHub / GitHub Pages	Quellcode, Versionshistorie und Veröffentlichung der statischen Website.
Browser	Datei lesen, Regeln anwenden, Eingaben bearbeiten, Freigabe und Bericht anzeigen.
Supabase Data API	HTTPS-Zugang zu genau definierten PostgreSQL-Funktionen (RPC).
PostgreSQL	Erneute Datenprüfung, transaktionales Speichern, Zugriffsschutz und SQL-Kennzahlen.

Wichtig: GitHub Pages führt kein serverseitiges Node.js oder PHP aus. Die Datenbank liegt separat bei Supabase. Nur der öffentliche Publishable-Key steht im Website-Code; Secret-Keys und Datenbankpasswörter gehören dort niemals hinein.

HOW-TO / 03

So ist das Projekt aufgebaut

```
manlin65.github.io/
  index.html                Homepage und Projektverweis
  assets/                   Logo, Schriften, gemeinsames Layout
  datencockpit/
    index.html              Oberfläche unter /datencockpit/
    styles.css              Auf das Cockpit begrenzte Styles
    app-source.js           Editierbarer Browser-Quelltext
    app.js                  Erzeugtes Browser-Bundle
    data.cjs                CSV, Normalisierung, Prüfregeln
    supabase.cjs            Geschützte RPC-Aufrufe
    config.js               Öffentliche Projektkonfiguration
    beispieldateien.zip     Fünf Testdateien
    downloads/              PDF und vollständiges Beispielpaket
  supabase/migrations/      Tabellen, Rechte und SQL-Funktionen
  scripts/                  Build, Entwicklungsserver, Prüfungen
  tests/                    Node-, SQL- und Clienttests
```

Technische Grundlage

Technik	Wofür?
HTML / CSS / JavaScript	Statische Oberfläche im bestehenden Homepage-Layout.
read-excel-file	Liest das erste Tabellenblatt einer XLSX-Datei.
esbuild	Bündelt Quellcode und Bibliothek in app.js.
Supabase / PostgreSQL	API, geprüfte Speicherung und SQL-Berichte.
Playwright / Chromium	Prüft den Ablauf im echten Browser.
PGlite	Lokale PostgreSQL-Engine für SQL- und Rechteprüfungen.

Der Homepage-Stil bleibt erhalten: originales ML-Logo, lokal gehostete Schriften Manrope und Source Sans 3, Petrolfarben, Navigation und Fußzeile. app.js wird erzeugt; Änderungen gehören in app-source.js und werden anschließend neu gebaut.

HOW-TO / 04

Bedienung: vom Upload zur Freigabe

1. Datei wählen

Die Seite öffnen und „Datei auswählen“ oder „Beispieldaten laden“ verwenden. Unterstützt werden CSV und XLSX; alte XLS-Dateien vorher als XLSX speichern. Das ZIP zuerst entpacken, dann eine einzelne Datei hochladen.

2. Ergebnis prüfen

Die Übersicht zählt Gesamtzeilen, gültige Zeilen, auffällige Zeilen und Ausschlüsse. „Nur Auffälligkeiten“ blendet fehlerfreie Zeilen aus. Die sichtbare Zeilennummer orientiert sich an den eingelesenen Daten inklusive Kopfzeile.

3. Werte berichtigen

Kundennamen, Datum, Betrag oder Status direkt in der Tabelle bearbeiten. Änderungen mit Tab oder einem Klick außerhalb des Felds übernehmen. Nach jeder Änderung muss die Freigabe erneut bestätigt werden.

4. Dubletten klären

Bei identischen Auftragsnummern sind beide Zeilen markiert. Die falsche Zeile ausschließen oder eine tatsächlich falsche Nummer korrigieren. Der Vergleich unterscheidet Groß- und Kleinschreibung nicht.

5. Ausdrücklich freigeben

Nur eine Auswahl ohne verbleibende Auffälligkeiten lässt sich freigeben. Das Kontrollkästchen bestätigen und „Freigeben & Bericht erstellen“ wählen. Jetzt erst werden die ausgewählten Aufträge übertragen und gespeichert.

6. Bericht nutzen

Monatswerte und Diagramm ansehen, den Bericht als CSV exportieren oder aktualisieren. „Import aus der Datenbank löschen“ entfernt den aktuellen Import und alle zugehörigen Aufträge nach einer Bestätigung.

Ausgeschlossene Zeilen und die ursprüngliche Upload-Datei werden nicht in der Datenbank gespeichert. Die API prüft die übermittelten Daten erneut; Browserprüfungen allein reichen nicht aus.

HOW-TO / 05

Dateiformat und Prüfregeln

Spalte	Erwartet / Beispiel
Auftragsnummer	Pflichtwert, maximal 80 Zeichen; z. B. A-1001. Keine Dublette innerhalb des Imports.
Datum	TT.MM.JJJJ oder JJJJ-MM-TT; gültiges Kalenderdatum von 2000 bis 2100. Excel-Datumszellen werden unterstützt.
Kunde	Pflichtwert, maximal 160 Zeichen; in der Demo nur fiktive Namen.
Betrag	Euro, 0 bis 10.000.000; maximal zwei Nachkommastellen. Z. B. 1.234,56 oder 1234.56.
Status	Genau offen, bezahlt oder storniert. Schreibweise wird normalisiert.

Dateigrenzen

• Maximal 2 MB pro Datei und 1.000 Auftragszeilen; die API akzeptiert höchstens 1 MB JSON-Nutzdaten.

• CSV: UTF-8 empfohlen, Komma oder Semikolon als Trennzeichen. Die Beispiele verwenden UTF-8 mit BOM und Semikolon für Excel.

• XLSX: Nur das erste Tabellenblatt wird gelesen. Die Pflichtspalten müssen jeweils einmal vorhanden sein; ihre Reihenfolge ist frei.

Warum bleibt die Freigabe gesperrt?

Ein Haken bei der Zustimmung hebt Datenfehler nicht auf. Korrigieren oder entfernen Sie zuerst alle markierten Zeilen. Nach Korrekturen die Zustimmung erneut aktivieren. Mindestens ein gültiger Auftrag muss ausgewählt bleiben.

Prüfung versus fachliche Freigabe

Das Cockpit prüft Formate, Pflichtangaben und Dubletten. Es weiß nicht, ob ein Betrag fachlich stimmt oder ob ein Kunde tatsächlich existiert. Diese Entscheidung trifft die Person, die die Datei prüft und freigibt.

HOW-TO / 06

Datenbank: zwei Tabellen, ein Import

datencockpit_batches	Bedeutung
id · uuid · Primärschlüssel	Eindeutige Kennung eines freigegebenen Imports.
token_hash · text	SHA-256-Hash des zufälligen Import-Zugriffsschlüssels; nicht der Schlüssel selbst.
created_at · timestamptz	Zeitpunkt des Imports.
expires_at · timestamptz	Zugriffsende, standardmäßig nach 24 Stunden.

Beziehung: Ein Import hat mehrere Aufträge. datencockpit_orders.batch_id verweist auf datencockpit_batches.id. Beim Löschen des Imports werden seine Aufträge durch ON DELETE CASCADE mit gelöscht.

datencockpit_orders	Bedeutung
batch_id · uuid	Zugehöriger Import / Fremdschlüssel.
order_number · varchar(80)	Auftragsnummer; zusammen mit batch_id der Primärschlüssel. Ein zusätzlicher Index verhindert Dubletten unabhängig von Groß-/Kleinschreibung.
order_date · date	Auftragsdatum für die Monatszuordnung.
customer · varchar(160)	Fiktiver Kundenname.
amount_cents · bigint	Betrag als ganze Cent-Zahl, z. B. 12345 = 123,45 €.
status · text	Nur offen, bezahlt oder storniert.

Wo sehe ich gespeicherte Daten?

Supabase-Projekt öffnen → Table Editor → datencockpit_orders. Einträge entstehen nach erfolgreicher Freigabe. datencockpit_batches zeigt den zugehörigen Import. Tests löschen ihre eigenen Daten wieder; leere Tabellen nach einem Test sind daher normal.

```
SELECT order_number, order_date, customer, amount_cents, status
FROM public.datencockpit_orders
ORDER BY order_date, order_number;
```

Die Abfrage oben ist für den administrativen SQL-Editor gedacht. Die öffentliche Website darf die Tabellen nicht direkt lesen.

HOW-TO / 07

Geschützte Funktionen und SQL-Bericht

RPC-Funktion	Aufgabe
datencockpit_status()	Bestätigt, dass die SQL-Funktion über die Supabase-API erreichbar ist.
datencockpit_import(p_rows)	Prüft alle Aufträge erneut, erzeugt Import-ID und Zugriffsschlüssel und speichert atomar. Fehler rollen den gesamten Import zurück.
datencockpit_report(p_batch_id, p_token)	Prüft Schlüssel und Ablaufzeit; liefert Monatswerte und Gesamtsummen.
datencockpit_delete(p_batch_id, p_token)	Entfernt nur den Import mit passendem Schlüssel; Aufträge werden mit gelöscht.

Die Kennzahlen entstehen in PostgreSQL

```
SELECT to_char(order_date, 'YYYY-MM') AS monat,
       count(*) AS auftraege,
       sum(amount_cents) FILTER
         (WHERE status = 'bezahlt') AS umsatz_cent,
       count(*) FILTER
         (WHERE status = 'offen') AS offene_auftraege,
       sum(amount_cents) FILTER
         (WHERE status = 'offen') AS offen_cent
FROM public.datencockpit_orders
WHERE batch_id = 'HIER-IMPORT-UUID-EINSETZEN'::uuid
GROUP BY 1 ORDER BY 1;
```

Beispiel für die Logik der Auswertung. Die echte Berichtsfunktion prüft davor die Berechtigung und ersetzt leere Summen mit 0. Die Website sendet keine beliebigen SQL-Anweisungen, sondern ruft nur diese fest definierten Funktionen auf.

Zugriff und Aufbewahrung

Row Level Security ist auf beiden Tabellen aktiv. Den API-Rollen anon und authenticated sind direkte Tabellenrechte entzogen. Nur die vorgesehenen Funktionen sind freigegeben. Bericht und Löschung benötigen einen gültigen Import-Schlüssel.

Der Schlüssel liegt im sessionStorage des aktuellen Tabs. Nach Schließen des Tabs ist er normalerweise nicht mehr verfügbar; Browser-Sitzungswiederherstellung kann ihn erhalten. Nach 24 Stunden endet der Zugriff in jedem Fall. Abgelaufene Daten werden beim nächsten erfolgreichen Import physisch entfernt. Es gibt keinen automatischen Löschjob.

Dies ist ein Beispieldaten-Projekt ohne Mitarbeiterkonten. Für echte Geschäftsdaten fehlen insbesondere Benutzerverwaltung, Rollen, Missbrauchsschutz und ein verbindlicher Löschprozess.

HOW-TO / 08

Supabase einrichten und kontrollieren

1. Kostenloses Projekt anlegen

Im Supabase-Dashboard ein eigenes Projekt anlegen, mit einem passenden Projektnamen. Eine EU-Region wählen. Das Datenbankpasswort sicher speichern und nicht in GitHub oder Website-Dateien übernehmen.

2. Tabellen und Rechte einrichten

SQL Editor öffnen. Zuerst 202610220001_datencockpit_tables.sql aus dem Paket vollständig ausführen. Danach 202610220002_datencockpit_functions.sql ausführen. Beide Dateien liegen zusätzlich unter supabase/migrations im Repository.

3. Auf Originalsprache achten

Automatische Browser-Übersetzung ausschalten, bevor SQL kopiert oder ausgeführt wird. Befehle müssen SELECT, CREATE und GRANT heißen, nicht „WÄHLE“, „ERSTELLEN“ oder übersetzte Varianten.

4. Öffentliche Konfiguration übernehmen

Project Settings → Data API: Project URL kopieren. Unter API Keys den Publishable-Key verwenden (sb_publishable_…). Nur diese öffentlichen Werte in datencockpit/config.js eintragen.

5. Verbindung prüfen

Im SQL-Editor SELECT public.datencockpit_status(); ausführen. Die Ausgabe enthält connected: true. Danach auf der Website „Verbindung prüfen“ verwenden. Der Test über die Website prüft zusätzlich Projektadresse und öffentlichen Schlüssel.

```
window.DATENCOCKPIT_CONFIG = Object.freeze({
  provider: 'supabase',
  supabaseUrl: 'https://DEIN-PROJEKT.supabase.co',
  supabasePublishableKey: 'sb_publishable_DEIN_OEFFENTLICHER_KEY'
});
```

Die aktuelle Projektadresse ist ebjoamsxkjufmilzabyp.supabase.co. Ein Secret- oder service_role-Key darf niemals in diese Datei. Der Client weist solche Schlüssel zurück.

HOW-TO / 09

Lokal entwickeln und auf GitHub veröffentlichen

Vorbereitung auf dem Entwicklungsrechner

Node.js ab Version 22 installieren. Repository klonen bzw. vorhandenen Checkout verwenden. Im Projektverzeichnis ausführen:

```
npm ci --include=dev
npm run build
npm test
npm run test:datencockpit:supabase
npm run dev:datencockpit
```

In der Codex-Cloud bei npm zusätzlich --cache /workspace/.npm-cache verwenden. Der Entwicklungsserver läuft auf Port 8888; der aktive Supabase-Modus braucht keine lokale Datenbank und keine .env-Datei. Die Browserprüfung benötigt Chromium; CHROMIUM_PATH kann den Browserpfad vorgeben.

Was muss nach GitHub?

Nach Quelltextänderungen immer neu bauen. app-source.js ist der Quelltext; app.js ist das fertige Browser-Bundle. Für dieses Projekt wird auch das erzeugte app.js committed, weil GitHub Pages den Branch direkt als statische Website veröffentlicht. Nur geprüfte Projektdateien hinzufügen; keine .env-Dateien oder Geheimschlüssel.

```
git status
git add datencockpit/ scripts/ supabase/ package.json package-lock.json
git diff --cached
git commit -m "DatenCockpit aktualisieren"
git push origin main
```

GitHub Pages einmalig einstellen

Repository → Settings → Pages → Build and deployment: **Deploy from a branch** auswählen. Branch **main**, Ordner **/ (root)**. Die bestehende Custom Domain lautet **www.mlinner-software.de**; die Datei CNAME im Repository bleibt erhalten. HTTPS verwenden.

Veröffentlichung überprüfen

Unter Actions den Lauf „pages build and deployment“ prüfen. Erst nach erfolgreichem Deploy die öffentliche URL öffnen und mit Strg+F5 aktualisieren. Auf der Website Verbindung, Beispieldatei, Freigabe und Bericht testen. Die gesamte Homepage wird mit demselben Branch veröffentlicht; andere Projekte nicht versehentlich überschreiben.

Netlify und Azure sind für die aktive Lösung nicht erforderlich. Alte Adapter im Repository dürfen bestehen bleiben. Die aktive Auswahl steht in datencockpit/config.js auf provider: supabase.

HOW-TO / 10

Fünf Dateien zum Ausprobieren

Alle Beispiele enthalten ausschließlich fiktive Aufträge. Entpacken Sie das ZIP und laden Sie eine einzelne CSV- oder XLSX-Datei hoch. Beide Excel-Dateien verwenden echte Excel-Datums- und Zahlenzellen.

Datei	Zeilen	Auffällig	Zweck
01_saubere_auftraege.csv	12	0	Direkt freigeben; vier Monate und alle drei Status.
02_dubletten.csv	8	4	Zwei Dublettengruppen; Groß-/Kleinschreibung testen.
03_fehlende_und_ungueltige_werte.csv	10	6	Pflichtwerte, Datum, Betrag und Status korrigieren.
04_monatsreport_2026.xlsx	24	0	Zwölf Monate mit je einem bezahlten und offenen Auftrag.
05_gemischte_pruefliste.xlsx	12	5	Excel-Import mit Dublette, fehlendem Kunden, Datum und negativem Betrag.

Beispiel 01 und 04: direkt freigeben

Datei laden, Ergebnis prüfen, Zustimmung aktivieren und Bericht erstellen. Für diese sauberen Dateien müssen keine Werte geändert werden.

Beispiel 02: Dubletten entfernen

Die zweite D-1002-Zeile ausschließen (Dateizeile 4). Die zweite Variante der D-1003-Auftragsnummer ausschließen (Dateizeile 6, als d-1003 geschrieben). Danach bleiben 6 gültige Aufträge.

So unterscheiden sich Fehler- und Zeilenzahl

„4 auffällige Zeilen“ bedeutet bei Beispiel 02 nicht vier zusätzliche Aufträge: Jede der zwei Dublettengruppen markiert beide betroffenen Zeilen. Durch Ausschluss jeweils einer Kopie werden auch die verbleibenden Originalzeilen gültig.

Dateien auch direkt in der PDF

Die fünf Originaldateien sind zusätzlich als PDF-Anhänge eingebettet. Viele Desktop-PDF-Reader zeigen sie im Bereich „Anhänge“ / Büroklammer. Browser-PDF-Viewer unterstützen das nicht immer. Das ZIP auf der Projektseite ist deshalb der verlässlichere Downloadweg.

HOW-TO / 11

Korrekturübungen und erwartete Ergebnisse

Beispiel 03: diese sechs Werte berichtigen

Auftrag	Korrektur
F-2002	Kunde ergänzen, z. B. Musterbetrieb.
F-2003	Datum ergänzen: 2026-10-03.
F-2004	2026-02-30 auf 2026-02-28 korrigieren.
F-2005	Betrag -50,00 auf 50,00 korrigieren.
F-2006	Betrag 125,999 auf 126,00 korrigieren.
F-2007	Status in Bearbeitung auf offen ändern.

Beispiel 05: Auswahl und Werte klären

Die zweite P-3002-Zeile ausschließen (Dateizeile 4). Bei P-3004 den Kunden ergänzen, z. B. Musterbetrieb. Bei P-3005 das Datum auf 2026-10-09 ändern. Bei P-3006 den Betrag von -45 auf 45 ändern. Danach bleiben 11 gültige Aufträge.

Kontrollwerte nach genau diesen Korrekturen

Beispiel	Aufträge	Bezahlt	Offen
01 · unverändert	12	2.802,00 €	3.102,00 €
02 · zwei Kopien ausgeschlossen	6	3.620,50 €	1.400,00 €
03 · wie oben korrigiert	10	2.696,00 €	2.200,00 €
04 · unverändert	24	21.789,00 €	6.120,00 €
05 · wie oben korrigiert	11	7.725,00 €	1.925,50 €

Die Beträge „Offen“ sind offene Euro-Volumen, keine Auftragsanzahlen. Stornierungen zählen bei „Aufträge“ mit, aber nicht bei den beiden Geldsummen. Werden andere Korrekturen vorgenommen oder weitere Zeilen ausgeschlossen, ändern sich die Kontrollwerte.

Schneller Vortrag mit den eingebauten Beispieldaten

„Beispieldaten laden“ auf der Seite verwendet einen separaten 8-Zeilen-Datensatz. Die zweite A-1003-Zeile ausschließen und den Kunden bei A-1004 ergänzen. Ergebnis: 7 Aufträge, 4.650,00 € bezahlt und 1.490,50 € offen. Diese Werte gehören nicht zu Datei 01 des ZIPs.

HOW-TO / 12

Hilfe, Abnahme und nächste Schritte

Beobachtung	Was tun?
Freigabe-Button ist grau	Alle verbleibenden Auffälligkeiten korrigieren oder ausschließen. Mindestens eine Zeile auswählen und Zustimmung erneut aktivieren.
SQL enthält übersetzte Wörter	Browser-Übersetzung abschalten, Original-SQL erneut kopieren und vollständig ausführen.
Verbindung schlägt fehl	Project URL und öffentlichen Publishable-Key prüfen. Beide SQL-Skripte vollständig ausführen. Supabase-Projektstatus kontrollieren.
Neuer Code erscheint nicht	GitHub-Actions-Deployment abwarten und die Seite mit Strg+F5 neu laden. Bei Bedarf Browsercache prüfen.
Import nicht gefunden / abgelaufen	Richtigen Browser-Tab nutzen. Nach 24 Stunden Datei erneut importieren. Ohne gespeicherten Import-Schlüssel kann der Bericht nicht geöffnet werden.
Tabellen sind leer	Es wurde noch nichts freigegeben, der Import wurde gelöscht oder nur ein automatischer Test lief. Tests räumen ihre eigenen Daten auf.
Excel wird nicht angenommen	XLSX statt XLS verwenden; erste Tabelle, Pflichtüberschriften, Dateigröße und Zeilenlimit kontrollieren.

Nachweisbarer Prüfstand

Zum dokumentierten Stand bestanden Build und 38 Node-Tests. Zusätzlich wurden die reale Supabase-Verbindung, gesperrte direkte Tabellenzugriffe, gültige und fremde Import-Schlüssel, SQL-Kennzahlen und Löschung geprüft. Der vollständige CSV- und XLSX-Ablauf wurde auf der veröffentlichten Website mit Chromium ausgeführt. Eigene Testimporte wurden danach entfernt.

Checkliste nach einer Änderung

- Build und Tests erfolgreich; erzeugtes app.js aktualisiert.
- Beide SQL-Skripte eingerichtet; Statusfunktion erreichbar.
- GitHub-Pages-Deployment erfolgreich; öffentliche Website neu geladen.
- Beispieldatei geprüft, freigegeben, Bericht und CSV-Export kontrolliert.
- Testimport gelöscht; keine Geheimschlüssel oder realen Kundendaten veröffentlicht.

Projektseite und Downloads

Diese Anleitung beschreibt eine Portfolio-Demo und keinen fertig abgenommenen Mehrbenutzerbetrieb für echte Unternehmensdaten.